

Database Processing with Python





Database Fundamentals

- A database is an organized collection of related data.
  - Examples include student information, library books, employee records, banking transactions and product inventories.
- A Database Management System (DBMS) provides facilities for:
  - storing structured data
  - retrieving required records
  - enforcing data rules
  - controlling access
- Examples
  - MySQL, PostgreSQL, Oracle Database and SQLite.





Relational Databases

- A relational database stores data in tables.
  - Table: collection of related records
  - Row: one complete record
  - Column: one property of a record
  - Primary key: column that uniquely identifies a row
  - Query: request for data from a database
  - Schema: structure of the database
- Example columns:

  reg_no name department semester cgpa





Database Processing Workflow

- A database application normally follows these stages:
  - Create or select a database.
  - Create the required tables.
  - Establish a connection from the application.
  - Execute SQL statements.
  - Retrieve and process query results.
  - Close the connection.
- focuses on database creation, table creation and querying.





Introduction to SQL

- SQL stands for Structured Query Language.
- SQL command categories:
  - DDL defines database structures: CREATE, ALTER and DROP.
  - DQL retrieves stored data: SELECT.
  - DML changes stored records: INSERT, UPDATE and DELETE.
- SQL keywords are normally written in uppercase for readability.





Creating a Database and Table

```
CREATE DATABASE college;

CREATE TABLE student (
   reg_no VARCHAR(10) PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   department VARCHAR(20),
   semester INT,
   cgpa DECIMAL(4, 2)
);
```





Basic SELECT Queries

```
-- All columns
SELECT * FROM student;

-- Selected columns
SELECT reg_no, name, cgpa FROM student;

-- Distinct values
SELECT DISTINCT department FROM student;
```

SELECT retrieves rows from a table.

* represents all columns.

DISTINCT removes duplicate results.





Filtering Query Results

```
SELECT name, cgpa FROM student WHERE cgpa >= 8.0;

SELECT * FROM student  WHERE department = 'IT' AND semester = 3;

SELECT * FROM student WHERE department IN ('IT', 'CSE');
```

WHERE filters rows.

AND requires both conditions.

IN checks membership in a collection of values.





Sorting and Summarizing Data

```
SELECT name, cgpa FROM student ORDER BY cgpa DESC LIMIT 3;

SELECT department, COUNT(*) AS total, AVG(cgpa) AS average_cgpa
        FROM student  GROUP BY department;
```

ORDER BY sorts results.

LIMIT restricts the number of rows.

Aggregate functions summarize groups of records.





Introduction to MySQL

- MySQL is a relational database-management system.
- Characteristics:
  - client-server architecture
  - support for multiple databases and users
  - user authentication and access control
  - SQL-based database operations
  - common use in web and server applications
- A MySQL client sends SQL statements to the MySQL server.
- The server processes them and returns results.





Accessing the MySQL Server

```
# Open the MySQL client
mysql -u root -p

-- Display available databases
SHOW DATABASES;

-- Exit the MySQL client
EXIT;
```

```
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root -p
```





Creating a MySQL Database

```
CREATE DATABASE college;

CREATE DATABASE IF NOT EXISTS college;

USE college;

SELECT DATABASE();
```





Creating a MySQL Table

```
CREATE TABLE student (
   reg_no VARCHAR(10) PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   department VARCHAR(20) NOT NULL,
   semester INT,
   cgpa DECIMAL(4, 2)
);

SHOW TABLES;

DESCRIBE student;
```



SHOW TABLES lists the tables.

DESCRIBE displays column names, data types, keys and null settings.



Querying a MySQL Table

```
SELECT * FROM student;

SELECT reg_no, name, cgpa FROM student WHERE department = 'IT';

SELECT name, cgpa FROM student ORDER BY cgpa DESC LIMIT 5;
```



The same SELECT concepts apply in MySQL.

Assume that the student table already contains records.



Python–MySQL Connectivity

- A Python program requires a database driver to communicate with MySQL.

Python program
↓
MySQL Connector/Python
↓
MySQL server
↓
Database

- The connector establishes connections, sends SQL statements, retrieves results and reports database errors.



Import statement: import mysql.connector



Installing MySQL Connector

```
# Install from a terminal
pip install mysql-connector-python

# Verify from Python
import mysql.connector

print(mysql.connector.__version__)
```



Use the same Python environment for pip and the program.

The MySQL server must also be installed and running.



Connecting to the MySQL Server

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your_password"
)

if connection.is_connected():
    print("Connected to MySQL")

connection.close()
```





Creating a Database

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your_password"
)

cursor = connection.cursor()
cursor.execute(
    "CREATE DATABASE IF NOT EXISTS college"
)

print("Database created successfully")
cursor.close()
connection.close()
```





Core Cursor Methods

- execute(operation, params=None)
  - Runs a specific database query or command. It safely binds parameters to protect against SQL injection.
- fetchone()
  - Retrieves the next single row of a query result set.
- fetchmany(size)
  - Retrieves the next set of rows up to the specified size.
- fetchall()
  - Retrieves all remaining rows from the query result set as a list of tuples.
- close()
  - Closes the cursor to free up resources on the database server





Connecting to the College Database

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your_password",
    database="college"
)

cursor = connection.cursor()
print("Connected to the college database")

cursor.close()
connection.close()
```





Creating a MySQL Table

```
query = """
CREATE TABLE IF NOT EXISTS student (
    reg_no VARCHAR(10),
    name VARCHAR(50),
    department VARCHAR(20),
    semester INT,
)
"""

cursor.execute(query)
print("Student table created")
```





Querying MySQL through Python

```
cursor.execute("""
    SELECT reg_no, name, department, cgpa
    FROM student
    ORDER BY cgpa DESC
""")

records = cursor.fetchall()

for record in records:
    print(record)

cursor.close()
connection.close()
```

```
('24IT102', 'Divya', 'CSE')
('24IT101', 'Arun', 'IT')
```





Introduction to SQLite and PySQLite

- SQLite is an embedded relational database.
- Characteristics:
  - stores a database in a local file
  - requires no separate database server
  - requires no database account
  - supports many standard SQL operations
  - suits local applications and classroom exercises
- PySQLite originally provided Python access to SQLite.
- Current Python versions provide this interface through the standard-library sqlite3 module.





Creating an SQLite Database

```
import sqlite3

connection = sqlite3.connect("college.db")

print("Database opened successfully")

connection.close()
```

If college.db exists, SQLite opens it.

Otherwise, SQLite creates the database file.

For a temporary database: sqlite3.connect(":memory:")



Creating an SQLite Table

```
import sqlite3

connection = sqlite3.connect("college.db")
cursor = connection.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS student (
        reg_no TEXT PRIMARY KEY,
        name TEXT NOT NULL,
        department TEXT,
        semester INTEGER,
        cgpa REAL
    )
""")

connection.commit()
connection.close()
```



Querying an SQLite Table

```
cursor.execute("""
    SELECT reg_no, name, department, cgpa
    FROM student
    ORDER BY cgpa DESC
""")

records = cursor.fetchall()

for reg_no, name, department, cgpa in records:
    print(f"{reg_no:<12}{name:<20}"
          f"{department:<12}{cgpa:>6.2f}")
```



```
24IT102    Divya          CSE          9.10
24IT101    Arun           IT           8.25
```



Parameterized SQLite Query

```
department = input("Enter department: ")

cursor.execute("""
    SELECT reg_no, name, cgpa
    FROM student
    WHERE department = ?
    ORDER BY cgpa DESC
""", (department,))

records = cursor.fetchall()

for record in records:
    print(record)
```

SQLite uses ? placeholders.

MySQL Connector uses %s placeholders.

Pass values separately from the SQL statement.





Key Takeaways

- Relational databases organize related data into tables.
- SQL CREATE statements define databases and tables.
- SELECT retrieves records from a table.
- WHERE filters rows, while ORDER BY sorts results.
- MySQL operates through a database server.
- Python uses MySQL Connector/Python to access MySQL.
- SQLite stores its database in a local file.
- Python provides SQLite access through sqlite3.
- MySQL Connector uses %s placeholders.
- SQLite uses ? placeholders.
- Parameterized queries should handle external values.

